

MIDISTRUCT — Description Technique Complète

Générateur MIDI Algorithmique pour REAPER

Developed by Acrosonus Mastering — acrosonus@outlook.com

Licence : GNU GPL v3 — <https://www.gnu.org/licenses/gpl-3.0.html>

Présentation

MIDISTRUCT est un script ReaScript Lua de composition algorithmique complète pour REAPER. À partir d'une progression d'accords et d'un style musical, il génère en quelques secondes un arrangement structuré de 3 à 4 minutes sur 5 pistes MIDI séparées, avec régions colorées, items nommés, automation CC et rapport texte. Ce n'est pas un arpégiateur ni un générateur de patterns aléatoires. Chaque décision musicale repose sur des règles de composition réelles, calibrées sur l'analyse de 25 hits pop/R&B/rock analysés note par note.

Ce que MidiStruct crée automatiquement dans REAPER

- 5 pistes MIDI colorées : DRUMS, BASS, PAD-CHORDS, MELODY, COUNTER-MEL
 - Items MIDI nommés par section sur chaque piste
 - Régions colorées dans la timeline (Intro, Verse, Chorus 1/2/3, Bridge, Outro...)
 - Automation CC#11 Expression sur le Pad (swell exponentiel)
 - Automation CC#1 Modulation sur la Melody (vibrato sur notes longues)
 - Tempo automation (Bridge -2 BPM, Chorus final +1 BPM)
 - Rapport texte avec seed, tonalité, structure et description du hook
-

Architecture du code

Le script est organisé en couches strictement séparées :

RNG → Générateur pseudo-aléatoire déterministe (LCG) Humanizer → Timing, vitesse, rubato, push & pull Engine → Moteur musical complet REAPER layer → Écriture MIDI, régions, CC, rapport

Aucune variable globale mutable partagée entre les couches.

L'Engine est instancié une seule fois et porte tout l'état interne.



ACROSONUS

Un seul fichier Lua, aucune dépendance externe.

1. RNG — Générateur Congruentiel Linéaire (LCG)

Problème résolu

Lua standard utilise `math.random` qui s'appuie sur le RNG C de la plateforme, non reproductible entre sessions. Il fallait un RNG embarqué et déterministe.

Solution : LCG de Lehmer (constantes ANSI C)

```
seed_n+1 = (seed_n × 1103515245 + 12345) mod 2^31  
rand() = seed_n+1 / 2^31 ∈ [0, 1)
```

Période = $2^{31} = 2\,147\,483\,648$. Même seed → même résultat garanti,

toutes plateformes, toutes sessions.

Distribution gaussienne — Box-Muller

```
Z = sqrt(-2 × ln(U1)) × cos(2π × U2)
```

Appliqué à la vitesse, au timing et au rubato pour produire des

micro-variations statistiquement réalistes autour d'une valeur centrale.

Tirage pondéré — Roulette wheel

`RNG:choices(seq, weights)` accumule les poids jusqu'à franchir un seuil

aléatoire. Utilisé partout où des choix musicaux ont des probabilités différentes.

Fix seed V4

Quand l'utilisateur entre 0, le seed est calculé via `reaper.time_precise() * 1000 mod 2147483647`

avant d'instancier l'Engine, de sorte qu'il est toujours loggé dans le rapport

et la popup de confirmation. Le rapport n'affiche plus jamais 0.

2. Humanizer — Modélisation du jeu humain

Timing : bruit gaussien contrôlé par tightness

```
σ = 5.5 / tightness  
timing = floor(base_tick + gauss(0, σ))
```

tightness : 0.6 (Lo-Fi, très lâche) → 2.0 (Techno, quasi-quantisé).

Rubato : décalage systématique par position dans la mesure

Step 15 (fin de phrase) → $\text{gauss}(-6, 2.0) \times \text{intensity}$ (anticipation forte)
Step 7 (demi-mesure) → $\text{gauss}(-3, 1.5) \times \text{intensity}$ (anticipation légère)
Step ×4 (temps forts) → $\text{gauss}(+2, 1.0) \times \text{intensity}$ (retard naturel)



Asymétrie qui reproduit le comportement naturel d'un musicien qui anticipe

les fins de phrase et appuie légèrement sur les temps forts.

Push & Pull — Décalage global par section

Chorus : -12 ticks (push — urgence, énergie) PreChorus : -6 ticks (push léger) Verse : +10 ticks (pull — laid-back, décontracté) Bridge : +8 ticks (pull léger)

Appliqué à l'ensemble des notes de mélodie et contre-mélodie de la section.

Technique utilisée consciemment par les producteurs R&B et pop depuis les années 80.

3. Inférence de tonalité — Vote par pitch class

Algorithme

Chaque pitch class (0–11) est comptabilisé sur tous les accords de la progression.

Les roots d'accords comptent double (poids 2 vs 1 pour les autres notes).

La root globale = pitch class avec le score le plus élevé.

Le mode est déterminé par la présence de la tierce mineure (root+3) :

Pop → majeur par défaut si ambiguïté Trap, House, Lo-Fi → mineur par défaut R&B → Dorien (mineur avec sixte majeure) Techno → Phrygien (mineur avec seconde mineure)

Gamme globale

Construite sur les octaves 3 à 6 (MIDI 48–84), dédoublée, triée.

Toutes les notes de mélodie en sortent — sauf les notes de passage chromatiques intentionnelles qui sont hors-gamme par définition.

4. Voice Leading — Minimisation du mouvement des voix

Pour les pads, chaque voix se déplace le moins possible entre deux accords.

pour chaque pc dans chord_pcs : $\text{best_note} = \text{argmin}_{\{\text{oct} \in [4,6]\}} |\text{pc} + \text{oct} \times 12 - \text{prev_voice}[i]|$

Contrainte d'espacement maximum de 14 demi-tons entre voix adjacentes

pour éviter les voicings ouverts non idiomatiques.

5. Hook — Intervalles Signature

Intervalles par style



Style	Intervalles	Caractère
Pop	7, 9, 5	Quinte, Sixte maj, Quarte
Lo-Fi	3, 5, 7	Tierce min, Quarte, Quinte
R&B	9, 7, 12	Sixte maj, Quinte, Octave
Techno	1, 6, 11	Seconde min, Triton, Septième
Rock	7, 5, 10	Quinte, Quarte, Septième min
Trap	3, 10, 7	Tierce min, Septième min, Quinte

Les intervalles chromatiques sont convertis en degrés de gamme via une table de mapping `iv_to_deg`, ce qui permet de construire des contours qui restent dans la gamme quelle que soit la tonalité inférée.

Patterns rythmiques avec syncopation intentionnelle

6 patterns rythmiques disponibles, avec des steps de syncopation délibérément placés (steps 3, 7, 11 — une double croche avant les temps forts).

Ces steps ont une probabilité 1.15× plus élevée d'être joués.

6. Règle de répétition ×3 inter-sections



Machine à états

`_hook_play_count` -- fois joué dans la variation courante `_hook_variation` -- niveau de variation (0=original, 1=+1deg, 2=+oct, 3=+oct+harm)

Le hook se répète 3 fois identique, puis avance au niveau suivant.

Les 4 niveaux de variation :

- **0** : hook original
- **1** : transposé +1 degré dans la gamme
- **2** : transposé +1 octave
- **3** : octave + harmonisation à la tierce sur les temps pairs

7. Répétition intra-phrase

Principe (la lacune la plus importante corrigée)

Dans un vrai hit, les mesures 1 et 2 d'un groupe de 4 sont souvent **identiques**.

La variation n'arrive qu'en mesure 3. C'est ce qui crée la mémorabilité.

Implémentation par cache

`phrase_role "A"` → génère la mélodie normalement, mémorise dans `_phrase_cache`
`phrase_role "Ap"` → retourne une copie exacte du cache (mesure 2 = mesure 1)
`phrase_role "A2"` → retourne le cache avec variation légère + note de résolution
`phrase_role "B"` → phrase contrastante (registre plus bas, longues notes)

La clé du cache est `section + chorus_index` pour distinguer Chorus 1/2/3.

```
-- Ap : copie exacte
if phrase_role=="Ap" and self._phrase_cache[cache_key] then
  for s, m in pairs(self._phrase_cache[cache_key]) do
    melody[s] = {note=m.note, dur=m.dur, ...}
  end
  return melody
end
-- A2 : variation légère (55% de chance de décaler d'1 degré vers la résolution)
if phrase_role=="A2" and self._phrase_cache[cache_key] then
  for s, m in pairs(cache) do
    if not m.is_motif and rng:rand()<0.55 then
      dir = (note < target_note) and 2 or -2
      note = nearest_scale_note(note+dir)
    end
    melody[14] = {note=target_note, is_resolution=true}
  end
  return melody
end
```

8. Durées pilotées par rôle

Problème résolu

Les durées aléatoires `choices({1,2,3,4}, {0.25,0.40,0.25,0.10})` ne tenaient

pas compte du rôle musical de la note. Un phrasé naturel a des durées cohérentes

avec la fonction de chaque note.

Fonction `role_dur`



```

local function role_dur(is_pass, is_strong, is_resol, is_mot) if is_pass then return 1
end -- Double croche fixe if is_resol then return
choices({3,4},{0.55,0.45}) end -- Blanche / ronde if is_mot then return
choices({2,3},{0.65,0.35}) end -- Noire pointée if is_strong then return
choices({2,3},{0.70,0.30}) end -- Noire return choices({1,2},{0.40,0.60})
- Crocheend

```

Appliqué à :

- Toutes les notes de passage chromatiques (is_pass=true → 1 step fixe)
- Toutes les notes du hook (is_mot=true → 2-3 steps)
- Toutes les notes de climax et de temps forts (is_strong=true)
- Les notes de résolution de fin de phrase (is_resol=true → 3-4 steps)

9. Structure mélodique A-A'-B-A

Contours par section

Section	Contour	Description
Intro	flat	Stable
Verse	wave	$\sin(s \times 0.8) \times 3$
PreChorus	rise	Montée linéaire
Chorus	peak	Arc parabolique
Bridge	fall	Descente
Outro	fall	Résolution

Climax unique

Chaque Chorus et PreChorus force une note climax au step 10 :

```

climax = nearest_scale_note(target_note + signature_interval[1])
vel_boost = max(0, 1.0 - |step - 10| × 0.08)

```

Les notes proches du climax sont jusqu'à 25% plus fortes.

Notes de passage chromatiques



ACROSONUS

Sur les steps impairs : 20% de probabilité d'insérer une note hors-gamme

(±1 demi-ton depuis la note précédente). Durée fixée à 1 step par `role_dur`.

10. Walking Bass différenciée par section

Problème résolu

Dans un vrai arrangement pop/R&B, la basse est sparse au Verse (laisse de l'espace), s'anime au PreChorus, et est pleine au Chorus.

Trois niveaux

Sparse (Verse, Bridge, Intro, Outro) :

Root sur les temps 1 et 3 seulement + approach chromatique → Beaucoup de silence, la mélodie respire

Intermédiaire (PreChorus) :

Root + quinte, densité croissante → Prépare le drop du Chorus

Plein (Chorus) :

Pattern complet (root, quinte, tierce, septième, approach) + Octave haute (+12 MIDI) → Maximum d'énergie

Le call & response (ajout de notes dans les silences de mélodie) n'est

activé que sur les sections actives, pas sur les sections sparse.

11. Call & Response inter-pistes

Tableau de densité partagé

`density_map[0..15] = 0 ou 1`

`_gen_melody` marque les steps occupés à 1.

`_gen_counter_melody` cible préférentiellement les steps à 0 (silences).

`_gen_bass` enrichit les silences sur les bars impairs (probabilité 25%).

12. Dominantes secondaires

Avant chaque changement d'accord avec distance de root ≥ 3 demi-tons :

`sec_dom_root = (next_chord_root + 7) mod 12`

Inséré avec 35% de probabilité comme accord de passage court (type "7").

Crée des résolutions harmoniques locales qui fluidifient les transitions.



13. Automation CC — Swell exponentiel et vibrato

CC#11 Expression — Courbe de premier ordre (pad)

Un point CC#11 toutes les 60 PPQ (granularité : double croche) :

Phase attack (65% de la durée) :

$t = \text{step_index} / \text{attack_steps} \in [0, 1]$
 $\text{exp_val} = 1 - e^{(-3t)}$
 $\text{val} = \text{start} + (\text{peak} - \text{start}) \times \text{exp_val}$

Modélise la réponse d'un système du premier ordre — identique au comportement physique d'un swell de cordes ou d'un pad analogique.

Phase release (35% de la durée) : descente linéaire vers 82% du pic.

CC#1 Modulation — Enveloppe vibrato

Sur les notes mélodiques ≥ 2 steps (480 PPQ) :

$t = \text{ppq} + \text{dur}/3 \rightarrow \text{CC\#1} = 0$ (onset vibrato)
 $t = \text{ppq} + 2 \times \text{dur}/3 \rightarrow \text{CC\#1} = 18$ (Chorus) ou 10 (autres)
 $t = \text{note_end} - 20 \rightarrow \text{CC\#1} = 0$ (extinction)

Le vibrato ne commence qu'après 1/3 de la durée de la note —

comportement naturel d'un chanteur ou d'un violoniste.

Appel API REAPER (corrigé V4)

```
reaper.MIDI_InsertCC(take, false, false, ppq, 0xB0, 0, 11, val, false)--  
^ ^ ^ ^ ^  
chanmsg chan CC# val
```

14. Batterie — Hi-hat open/closed + Ride cymbal

Notes GM utilisées

Note	Instrumen t	Durée	Contexte
36	Kick	110 ticks	Patterns du style
38	Snare	115 ticks	Temps 2 et 4, ghost notes, fills
42	Hi-hat fermé	95 ticks	Steps hors open_step s
46	Hi-hat ouvert	220 ticks	Steps définis par



			le style
49	Crash	800 ticks	Début de chaque section
51	Ride	95 ticks	Lo-Fi et Bridge lofi
53	Ride bell	95 ticks	Lo-Fi, temps 1 et 3

Open_steps par style

Style	Open steps	Caractère
Pop	4, 12	Ouverture sur les 2 et 4
House	8	Mi-mesure
Rock	0,4,8,12	Permanent (feel live)
Trap	aucun	Toujours fermé

15. Densité mélodique — Calibrage sur analyse MIDI

Analyse de 50 hits de référence

Après analyse note par note des fichiers MIDI de référence

(Titanium, Numb, Shake It Off, Blinding Lights, ...) :

- Mélodie lead pop : **3 à 5 notes par mesure**
- Mélodie R&B : **5 à 7 notes par mesure**

Formule calibrée

$$\text{base_density} = 0.13 + \text{comp} \times 0.036 + (\text{is_chorus} ? 0.08 : 0)$$


ACROSONUS

Complexité	Densité	Notes/mesure approx
3-4	24-28%	~3-4 (Taylor Swift, Billie Eilish)
5	31%	~4-5 (Pop mid-tempo, EDM)
6-7	35-39%	~5-6 (R&B, Rock)
8-10	42-49%	~6-7 (Neo-Soul, Jazz)

16. Intégration REAPER

Couleurs des pistes (BGR + flag 0x1000000)

```
color = R + G×256 + B×65536 + 0x1000000reaper.SetMediaTrackInfo_Value(track, "I_CUSTOMCOLOR", color)
```

Régions colorées (isrgn=true)

Détection de changement via clé composite **section + chorus_index**

pour distinguer Chorus 1, Chorus 2, Chorus 3.

```
reaper.AddProjectMarker2(0, true, region_start, region_end, label, -1, color)
```

Items nommés par section

```
reaper.GetSetMediaItemTakeInfo_String(take, "P_NAME", "MELODY | Chorus 2", true)
```

MIDI_DisableSort / MIDI_Sort

Toutes les insertions après **MIDI_DisableSort(take)**.

MIDI_Sort(take) appelé une seule fois par take à la fin.

Divise le temps d'écriture par un facteur proportionnel au nombre de notes.



17. Structure de chanson générée

Intro (strip_level 2 : drums + pad seulement) Verse × 2 PreChorus × 1 Chorus 1 × 2 (hook original) Verse × 2 PreChorus × 1 Chorus 2 × 2 (hook +1 octave) Bridge × 1 (modulé +2 demi-tons, arrangement dépouillé) Chorus 3 × 2 (hook +octave + harmonisation tierce) Outro × 1 (strip-down)

Roles de phrases dans chaque groupe de 4 mesures : A → Ap → B → A2

18. Paramètres utilisateur

Dialogue 1 — Style

- Style principal (1-7 : Pop, Techno, Lo-Fi, R&B, House, Trap, Rock)
- Style secondaire pour mix hybride (0 = aucun)
- Mix % (interpolation linéaire des paramètres numériques)
- Complexité 1-10 (densité mélodique)
- Seed (0 = aléatoire, généré avant Engine:new et loggé)

Dialogue 2 — Preset ou Manuel

- 17 presets intégrés
 - 0 = saisie manuelle avec syntaxe **Note[Qualité][:Durée]**
-

19. Sortie typique

Session REAPER vierge, **Am:4 F:4 C:4 G:4**, Pop, complexité 5 :

5 pistes MIDI colorées~120 items MIDI nommés8 régions colorées (Chorus 1/2/3 distincts)3 marqueurs de tempo automatiquesCC#11 sur PAD (courbe exponentielle par accord)CC#1 sur MELODY (vibrato sur notes longues)MIDISTRUCT_rapport.txt dans le dossier projet

Compatibilité

- REAPER 6.x et 7.x (Windows, macOS, Linux)
 - Aucune dépendance externe
 - Compatible tous VSTi GM et non-GM
-

MIDISTRUCT — Acrosonus Mastering — GNU GPL v3



ACROSONUS